BIBLIOTECA WEB DE PDFS E DOCX

Lume

Banco de dados

Diagrama de relacionamentos, dicionário de dados e operação do PostgreSQL 16 e do Redis 7.

ESQUEMA	BANCO	MIGRAÇÕES	EDIÇÃO
11 tabelas · 3 apps Django	PostgreSQL 16 · Redis 7	accounts · library · processing 0001	1.1.0 · outubro de 2026

Clara A. Portes

Copyright © 2026 Clara A. Portes · Licença MIT

SUMÁRIO

Conteúdo

01 · Visão rápida 03

02 · Visão geral 04

- ÁREAS DO MODELO 04
- BANCO SUPORTADO E EXTENSÕES 04
- O QUE FICA NO DISCO 05

03 · Diagrama de relacionamentos 06

- CARDINALIDADES 07
- TABELAS DO FRAMEWORK 08

04 · Dicionário de dados e convenções 09

- CONVENÇÕES DE NOMES 09
- TIPOS USADOS 10
- VALORES FIXOS (ESCOLHAS) 10

05 · Integridade e concorrência 12

06 · Tabelas 14

- ACCOUNTS_USER 14
- ACCOUNTS_APITOKEN 15
- LIBRARY_DOCUMENT 15
- LIBRARY_DOCUMENTCONTENT 18
- LIBRARY_TAG 18
- LIBRARY_DOCUMENTTAG 19
- LIBRARY_COLLECTION 19
- LIBRARY_COLLECTIONITEM 20
- LIBRARY_READINGSTATE 20
- LIBRARY_ANNOTATION 21
- PROCESSING_JOB 22

07 · Máquinas de estado 24

08 · Busca em texto completo 26

09 · Redis: filas, chaves e eventos 28

- UM DOCUMENTO PASSO A PASSO 28
- CHAVES DO REDIS 29
- EVENTOS EM TEMPO REAL 29

10 · Consultas úteis para operação 31

11 · Como preparar o banco 35

12 · Manutenção, backup e restauração 37

- VACUUM, ANALYZE E ÍNDICES 37
- BACKUP E RESTAURAÇÃO 37
- PROBLEMAS COMUNS 38

13 · Evolução do esquema 40

14 · Histórico de versões 41

CAPÍTULO 01

Visão rápida

O esquema do Lume em números, e a divisão de responsabilidades entre o PostgreSQL, o Redis e o volume de arquivos.

11 tabelas do Lume `+ 6 tabelas do Django`	**3** apps com modelos `accounts · library · processing`	**111** colunas `33 só em library_document`	**14** chaves estrangeiras `todas obrigatórias`
47 índices `2 GIN · 1 parcial · 2 de expressão`	**12** restrições CHECK `11 de "≥ 0" + progresso`	**3** migrações `todas 0001_initial`	**2** streams `docs:extract · docs:describe`

Onde cada coisa mora

POSTGRESQL 16 · FONTE DA VERDADE

- Contas, papéis, preferências e tokens (só o hash).
- Documentos, metadados, sumário, descrição por IA e texto extraído.
- Etiquetas, coleções, estantes, progresso e anotações.
- Estado do pipeline e histórico de cada etapa.
- Vetor de busca e índices de trigramas.
- Sessões (`django_session`).

REDIS 7 · TRANSPORTE

- Filas de trabalho (Redis Streams) e grupo `workers`.
- Retentativas agendadas (`docs:delayed`).
- Avisos em tempo real (pub/sub `events:*`).
- Batimento dos workers (TTL 30 s).
- Bloqueio de login (TTL 15 min).
- Cache do Django, com cópia das sessões.

VOLUME /DATA · ARQUIVOS

- Original enviado, sem alteração.
- Capa em JPEG.
- HTML de leitura do DOCX, antes e depois da higienização.
- Imagens extraídas do DOCX.
- Envios em andamento (`tmp/*.part`).

> REGRA DE OURO
>
> Perder o Redis não perde dados: o worker recria os grupos e a varredura recoloca na fila o que estava pendente. Perder o PostgreSQL ou o volume `/data` perde dados. O backup começa por esses dois (capítulo 12).

Para quem é este documento

Para quem instala, opera ou desenvolve o Lume: a referência de cada tabela e coluna, as regras que o banco e a aplicação garantem, o uso do Redis e as tarefas de rotina (consultas de diagnóstico, manutenção, backup e restauração). Os nomes de tabelas, colunas, índices e chaves foram conferidos contra os modelos (`web/apps/*/models.py`) e as migrações iniciais, que são a fonte da verdade.

CAPÍTULO 02

Visão geral

O Lume guarda tudo o que importa no PostgreSQL. O Redis só transporta trabalho e avisos; os arquivos ficam num volume de disco compartilhado.

PostgreSQL é a fonte da verdade

Se o Redis for apagado, nada se perde: o worker recria os grupos das filas e uma varredura periódica encontra documentos parados no meio do caminho e os coloca na fila de novo. Se o processador Java ou um worker cair durante uma etapa, o estado gravado no banco diz exatamente o que falta fazer.

Áreas do modelo

DOMÍNIO	APP	TABELAS	O QUE GUARDA
CONTAS	`accounts`	`accounts_user`, `accounts_apitoken`	Contas, papéis, preferências e tokens pessoais da API.
BIBLIOTECA ACERVO	`library`	`library_document`, `library_documentcontent`	Documentos com metadados, capa, sumário, texto, descrição por IA e busca.
BIBLIOTECA ORGANIZAÇÃO	`library`	`library_tag`, `library_documenttag`, `library_collection`, `library_collectionitem`	Etiquetas e coleções.
BIBLIOTECA LEITURA	`library`	`library_readingstate`, `library_annotation`	Estante, página, progresso, favoritos e anotações de cada pessoa.
PROCESSAMENTO	`processing`	`processing_job`	Cada tentativa de cada etapa do pipeline, com duração e erro.
REDIS E EVENTOS	`core`, `realtime`	— (sem tabelas)	Filas, retentativas, avisos em tempo real, batimento e cache (capítulo 9).

Os apps `core`, `ai` e `realtime` não têm modelos. A IA (app `ai`) grava apenas em colunas de `library_document` (`summary*`, `language`) e em `library_documenttag` com origem `ia`.

Banco suportado e extensões

PostgreSQL 16 (imagem `postgres:16-alpine` do `docker-compose.yml`); versões 14 ou mais novas também funcionam. A primeira migração do app `library` cria o que a busca precisa:

OBJETO	TIPO	PARA QUÊ
`pg_trgm`	extensão	Similaridade por trigramas, para achar títulos mesmo com erro de digitação.
`unaccent`	extensão	Remove acentos ao indexar e ao buscar (decisões encontra decisoes).
`pt_unaccent`	configuração de busca textual	Cópia de `portuguese` com `unaccent` antes do radical português para `word`, `hword` e `hword_part`. Criada só se ainda não existir.

O que fica no disco

Os arquivos ficam no volume compartilhado `/data` (volume `documents` no Docker, `DATA_ROOT` na configuração), que o web, o worker e o processador Java montam. Cada documento tem uma pasta com o mesmo nome do seu `id`. O banco guarda só o caminho implícito (pelo `id`), nunca o binário.

CAMINHO	QUEM GRAVA	CONTEÚDO
`documents/<id>/original.pdf` ou `.docx`	web	O arquivo enviado, sem alteração.
`documents/<id>/cover.jpg`	processador	Capa (página 1 do PDF ou capa tipográfica do DOCX), 480 px de largura.
`documents/<id>/reader.html`	processador	HTML de leitura gerado do DOCX (antes da higienização).
`documents/<id>/reader.safe.html`	worker	O mesmo HTML depois da lista de permissões; é o que o navegador recebe.
`documents/<id>/media/`	processador	Imagens extraídas do DOCX.
`tmp/<uuid>.part`	web	Envio em andamento; apagado ao terminar ou falhar.

Excluir um documento pela aplicação apaga a pasta inteira e as linhas ligadas a ele; excluir uma conta faz o mesmo para todos os documentos dela.

CAPÍTULO 03

Diagrama de relacionamentos

As 11 tabelas do Lume e as 14 chaves estrangeiras que as ligam. Na página seguinte, o diagrama completo; depois, a lista de cardinalidades.

As ligações são chaves estrangeiras de verdade, todas obrigatórias (nenhuma aceita nulo). O pé de galinha fica do lado “muitos” e o traço duplo do lado “um”; o círculo marca o lado opcional da relação 1 : 0..1 entre o documento e o seu texto. Cada caixa mostra as colunas principais; as demais aparecem como “+ N colunas” e estão completas nas fichas do capítulo 6. A faixa colorida indica o domínio: CONTAS BIBLIOTECA PROCESSAMENTO.

Leitura rápida do desenho:

- **Contas.** Uma conta tem vários tokens. O token guarda só o hash; a autenticação calcula o SHA-256 do valor recebido e procura em `token_hash`.
- **Biblioteca.** O documento pertence a uma conta. O texto completo fica numa tabela à parte, ligada 1 : 0..1. Etiquetas e coleções são ligações N:N com tabelas intermediárias, que guardam dados da própria ligação (quem sugeriu a etiqueta; a posição na coleção).
- **Leitura.** Estado de leitura e anotações são da pessoa, não do documento: num documento compartilhado, cada leitor tem a sua estante, a sua página e as suas anotações. Por isso essas tabelas apontam ao mesmo tempo para a conta e para o documento.
- **Processamento.** Cada etapa de cada tentativa vira uma linha em `processing_job`.

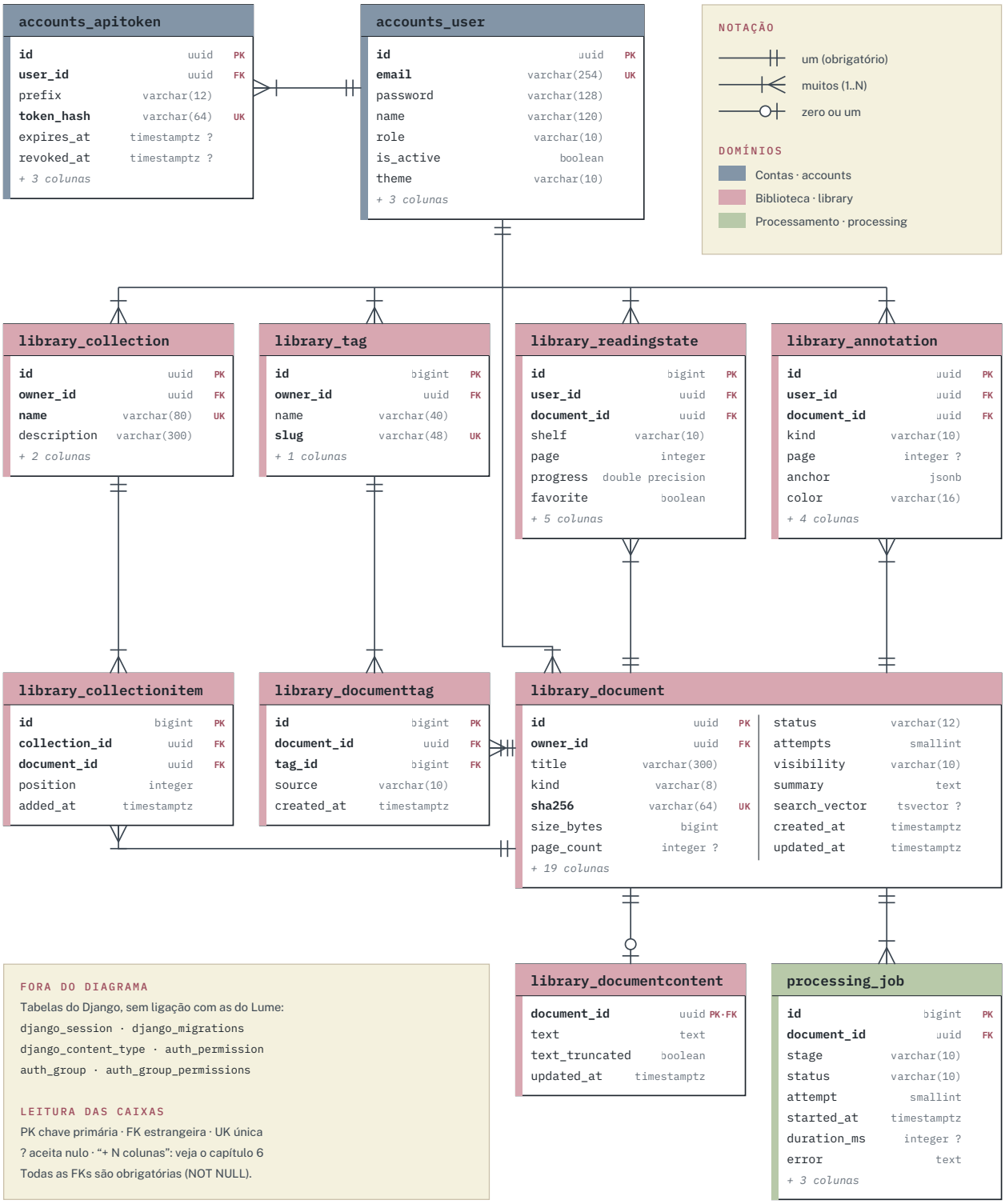


FIGURA 1 Todas as tabelas do Lume, com chaves primárias (PK), estrangeiras (FK) e únicas (UK). "?" marca coluna que aceita nulo.

Cardinalidades

TABELA "UM"	RELAÇÃO	TABELA "MUITOS"	PELA COLUNA
`accounts_user`	1 : N	`accounts_apitoken`	`user_id`
`accounts_user`	1 : N	`library_document`	`owner_id`
`accounts_user`	1 : N	`library_tag`	`owner_id`

TABELA “UM”	RELAÇÃO	TABELA “MUITOS”	PELA COLUNA
`accounts_user`	1 : N	`library_collection`	`owner_id`
`accounts_user`	1 : N	`library_readingstate`	`user_id`
`accounts_user`	1 : N	`library_annotation`	`user_id`
`library_document`	1 : 0..1	`library_documentcontent`	`document_id` (também PK)
`library_document`	1 : N	`library_documenttag`	`document_id`
`library_tag`	1 : N	`library_documenttag`	`tag_id`
`library_collection`	1 : N	`library_collectionitem`	`collection_id`
`library_document`	1 : N	`library_collectionitem`	`document_id`
`library_document`	1 : N	`library_readingstate`	`document_id`
`library_document`	1 : N	`library_annotation`	`document_id`
`library_document`	1 : N	`processing_job`	`document_id`

Pelas restrições únicas: cada conta tem no máximo um estado de leitura por documento; cada documento aparece uma vez em cada coleção e recebe cada etiqueta uma vez.

Tabelas do framework

O Django cria e mantém estas tabelas; o Lume não grava nelas diretamente:

TABELA	USO
`django_session`	Sessões do navegador. O motor `cached_db` grava aqui e guarda uma cópia no cache do Redis.
`django_migrations`	Quais migrações já foram aplicadas.
`django_content_type`	Registro dos modelos, usado pelo sistema de permissões do Django.
`auth_permission`, `auth_group`, `auth_group_permissions`	Criadas pelo app `django.contrib.auth`. O Lume usa a coluna `role` da conta e não usa grupos; as tabelas ficam vazias, exceto as permissões padrão. Como a conta não herda `PermissionsMixin`, não existem tabelas de ligação entre contas e grupos.

CAPÍTULO 04

Dicionário de dados e convenções

Como nomes, chaves, tipos, datas e valores padrão são escolhidos — para ler qualquer tabela sem surpresa e escrever consultas sem adivinhar.

Convenções de nomes

OBJETO	PADRÃO	EXEMPLO
Tabela	`<app>_<modelo>`, minúsculas, sem acento nem separador no modelo	`library_documenttag`
Chave primária	`id`; exceção: `library_documentcontent.document_id`	`accounts_user.id`
Chave estrangeira	`<relação>_id`; `owner_id` para o dono, `user_id` para dados pessoais	`library_tag.owner_id`
Datas	sufixo `_at`, sempre `timestamptz`	`last_opened_at`
Contadores e medidas	sufixo da unidade ou `_count`	`size_bytes`, `duration_ms`, `page_count`
Marcadores booleanos	adjetivo ou `is_` / `has_`	`is_active`, `has_reader_html`, `favorite`
Índice nomeado	prefixo da tabela (às vezes abreviado) + propósito	`library_doc_owner_created`
Restrição única	`<tabela>_<colunas>_unique` ou `<tabela>_unique`	`library_tag_owner_slug_unique`
Índice automático do Django	`<tabela>_<coluna>_<hash>`; `…_like` para `varchar_pattern_ops`	`library_tag_slug_…_like`
Verificação “≥ 0”	`<tabela>_<coluna>_check` (nome dado pelo PostgreSQL)	`processing_job_attempt_check`

Nas fichas, “…” substitui o trecho de hash que o Django gera nos nomes de índices automáticos. Para ver o nome exato num banco: `\di library_tag*` no `psql`.

Regras do modelo

- **Chaves.** As entidades que aparecem em endereços e na API (contas, tokens, documentos, coleções e anotações) usam UUID gerado pela aplicação; ninguém consegue adivinhar o próximo. Tabelas de ligação e de histórico usam `bigint` com `GENERATED BY DEFAULT AS IDENTITY`.
- **Datas.** Sempre `timestamptz`, gravadas em UTC. A interface converte para o fuso de `APP_TIMEZONE` (padrão `America/Sao_Paulo`).
- **Textos opcionais** são `''` (vazio) e não nulo, como manda a convenção do Django. Nulo fica para números e datas que podem não existir (páginas de um arquivo ainda não lido, fim de uma etapa em andamento).
- **Valores padrão** (coluna “Padrão” das fichas) são aplicados pela aplicação ao criar o registro; o banco não tem `DEFAULT`. Ao inserir linhas à mão, informe todas as colunas não nulas.
- **Listas e objetos** ficam em `jsonb` com padrão `[]` ou `{}`, nunca nulo.

- **Escolhas** (estado, papel, estante, cor) são `varchar` com valores fixos validados pela aplicação; o banco não tem `CHECK` para elas.

Tipos usados

TIPO	CAMPO DJANGO	ONDE
`uuid`	`UUIDField`	Chaves de contas, tokens, documentos, coleções e anotações.
`bigint`	`BigAutoField`, `BigIntegerField`	Chaves de etiquetas, ligações, leituras e jobs; `size_bytes`.
`integer`	`PositiveIntegerField`	Páginas, palavras, medidas e versão da capa, posição, duração.
`smallint`	`PositiveSmallIntegerField`	`attempts` do documento e `attempt` do job.
`double precision`	`FloatField`	`progress` do estado de leitura.
`varchar(n)`	`CharField`, `EmailField`, `SlugField`	Textos curtos com limite: títulos, nomes, escolhas, hashes, cores.
`text`	`TextField`	Textos longos: descrições, texto extraído, notas, erros.
`boolean`	`BooleanField`	Marcadores (ativo, favorito, capa gerada…).
`jsonb`	`JSONField`	Sumário, metadados, âncoras, posição, preferências e detalhes das etapas.
`tsvector`	`SearchVectorField`	`search_vector` da busca em texto completo.
`timestamptz`	`DateTimeField`	Todas as datas.

Valores fixos (escolhas)

COLUNA	VALOR GRAVADO	RÓTULO NA INTERFACE
`accounts_user.role`	`admin` · `member`	Administrador · Membro
`accounts_user.theme`	`system` · `light` · `dark`	Sistema · Claro · Escuro
`library_document.kind`	`pdf` · `docx`	PDF · DOCX
`library_document.status`	`queued` · `extracting` · `describing` · `ready` · `failed`	Na fila · Lendo o documento · Escrevendo a descrição · Pronto · Falhou
`library_document.visibility`	`private` · `shared`	Só eu · Todos da instalação
`library_document.summary_source`	`local` · `anthropic` · `openai` (ou vazio)	—
`library_documenttag.source`	`ia` · `usuario`	IA · Usuário
`library_readingstate.shelf`	`''` · `para_ler` · `lendo` · `lido`	— · Quero ler · Lendo · Lidos
`library_annotation.kind`	`note` · `highlight` · `bookmark`	Nota · Destaque · Marcador
`library_annotation.color`	`salvia` · `rose` · `ardosia` · `camurca`	As cores da paleta do Lume
`processing_job.stage`	`extract` · `describe`	Leitura do arquivo · Descrição por IA

BANCO DE DADOS

COLUNA	VALOR GRAVADO	RÓTULO NA INTERFACE
`processing_job.status`	`running` · `succeeded` · `retrying` · `failed`	Rodando · Concluída · Vai tentar de novo · Falhou

CAPÍTULO 05

Integridade e concorrência

O que o banco garante sozinho, o que a aplicação garante, e como vários workers trabalham ao mesmo tempo sem pisar um no outro.

REGRA	COMO É GARANTIDA
Um e-mail por conta, sem diferenciar maiúsculas	Índice único em `lower(email)` e e-mail gravado em minúsculas.
Mesmo arquivo não entra duas vezes na biblioteca de uma pessoa	`UNIQUE (owner_id, sha256)`. O envio repetido devolve o documento existente com `duplicate: true`.
Uma etiqueta por nome e dono	`UNIQUE (owner_id, slug)`.
Nomes de coleção sem repetição	`UNIQUE (owner_id, lower(name))`.
Um estado de leitura por pessoa e documento	`UNIQUE (user_id, document_id)`.
Uma etiqueta por documento; um documento por coleção	`UNIQUE (document_id, tag_id)` e `UNIQUE (collection_id, document_id)`.
Progresso entre 0 e 1	`CHECK (progress >= 0 AND progress <= 1)`.
Contadores nunca negativos	`CHECK (… >= 0)` nas 11 colunas inteiras sem sinal (páginas, palavras, medidas da capa, tentativas, posição, duração).
Duas cópias do worker não processam o mesmo documento	A etapa começa com `SELECT … FOR UPDATE SKIP LOCKED` e só segue se o estado for o esperado; mensagem repetida ou atrasada vira “skipped”.
A fila só recebe o documento depois que a linha existe	O `XADD` é feito em `transaction.on_commit`, depois do `COMMIT`.
Retentativa publicada uma única vez	Ao vencer a espera, só o worker que consegue o `ZREM` em `docs:delayed` faz o `XADD`.
Etiquetas da IA não apagam as do usuário	Ao gravar etiquetas de origem `ia`, só as ligações `ia` são trocadas; etiquetas sem nenhum documento são removidas.

EXCLUSÕES EM CASCATA

As chaves estrangeiras são `DEFERRABLE INITIALLY DEFERRED` e não têm `ON DELETE CASCADE` no banco: quem apaga em cascata é o Django (`on_delete=CASCADE`). Excluir uma conta pela aplicação apaga os documentos, o texto, as etiquetas, as coleções, as leituras, as anotações, os jobs e as pastas no disco. Um `DELETE` feito direto no `psql` falha se houver linhas filhas; apague as filhas antes ou use a API/o painel.

Ordem das filhas, para um DELETE manual

Se for mesmo preciso apagar um documento direto no banco (por exemplo, numa cópia de teste), apague primeiro as linhas que apontam para ele, numa transação:

```
BEGIN;
DELETE FROM processing_job          WHERE document_id = :id;
DELETE FROM library_annotation      WHERE document_id = :id;
DELETE FROM library_readingstate    WHERE document_id = :id;
DELETE FROM library_collectionitem  WHERE document_id = :id;
DELETE FROM library_documenttag     WHERE document_id = :id;
DELETE FROM library_documentcontent WHERE document_id = :id;
DELETE FROM library_document        WHERE id = :id;
COMMIT;
```

A pasta `/data/documents/<id>/` continua no disco e precisa ser removida à parte. Pela interface ou pela API, tudo isso é feito de uma vez.

CAPÍTULO 06

Tabelas

Uma ficha por tabela, na ordem dos domínios. “Nulo” indica se a coluna aceita `NULL`; “Padrão” é o valor que a aplicação grava ao criar o registro.

FICHA 01 CONTAS ACESSO — 10 colunas · 2 índices

accounts_user

Contas de quem usa a instalação. O login é feito pelo e-mail; o papel define quem administra.

Referenciada por `accounts_apitoken.user_id`, `library_document.owner_id`, `library_tag.owner_id`, `library_collection.owner_id`, `library_readingstate.user_id` e `library_annotation.user_id`.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	uuid	não	uuid4()	Identificador da conta, gerado pela aplicação.
email UK	varchar(254)	não	—	E-mail de acesso, gravado em minúsculas. Único sem diferenciar maiúsculas (índice sobre lower(email)).
password	varchar(128)	não	—	Hash da senha no formato do Django (PBKDF2-SHA256 com sal e iterações). A senha em si nunca é gravada.
last_login	timestamptz	sim	—	Último login pela tela de entrada (coluna herdada de AbstractBaseUser).
name	varchar(120)	não	—	Nome exibido no menu, nos documentos compartilhados e no painel.
role	varchar(10)	não	'member'	admin ou member. Administradores veem o painel Sistema e gerenciam contas.
is_active	boolean	não	true	Conta desativada não entra, não usa tokens e é desconectada do WebSocket.
theme	varchar(10)	não	'system'	system, light ou dark.
reader_prefs	jsonb	não	{}	Preferências do leitor (cor da página e tamanho da letra). A API recusa objetos com mais de 2.000 caracteres.
created_at	timestamptz	não	now()	Criação da conta.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
accounts_user_pkey	PRIMARY KEY (id)	Chave primária.
accounts_user_email_unique	UNIQUE INDEX (lower(email))	Impede duas contas com o mesmo e-mail escrito de formas diferentes.

EXEMPLOS DE JSON

reader_prefs Gravado pelo leitor quando a pessoa troca a cor da página ou o tamanho da letra.

```
{"theme": "paper", "fontSize": 19}
```

FICHA 02 CONTAS ACESSO — 9 colunas · 6 índices

accounts_apitoken

Tokens pessoais para scripts e integrações (cabeçalho `Authorization: Bearer`). Só o hash é gravado; o valor completo aparece uma única vez, na criação.

Aponta para `user_id → accounts_user.id`.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	uuid	não	uuid4()	Identificador do token.
user_id FK	uuid	não	—	Dono do token → accounts_user.id.
name	varchar(80)	não	—	Para que o token serve (ex.: importação noturna).
prefix	varchar(12)	não	—	8 caracteres hexadecimais visíveis no token (dl_<prefixo>_<segredo>) e na tela, para reconhecê-lo.
token_hash UK	varchar(64)	não	—	SHA-256 do token completo. A autenticação procura por este campo.
created_at	timestamptz	não	now()	Criação.
last_used_at	timestamptz	sim	—	Último uso; atualizado no máximo uma vez por minuto.
expires_at	timestamptz	sim	—	Validade opcional.
revoked_at	timestamptz	sim	—	Preenchido ao revogar. O registro fica para histórico.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
accounts_apitoken_pkey	PRIMARY KEY (id)	Chave primária.
accounts_apitoken_token_hash_key	UNIQUE (token_hash)	Busca do token na autenticação.
accounts_apitoken_token_hash_…_like	btree (token_hash varchar_pattern_ops)	Criado pelo Django para buscas com LIKE.
accounts_apitoken_prefix_…	btree (prefix)	Consulta pelo prefixo exibido.
accounts_apitoken_prefix_…_like	btree (prefix varchar_pattern_ops)	Criado pelo Django para buscas com LIKE.
accounts_apitoken_user_id_…	btree (user_id)	Tokens de uma conta (chave estrangeira).

FICHA 03 BIBLIOTECA ACERVO — 33 colunas · 9 índices

library_document

Um arquivo enviado (PDF ou DOCX) com tudo o que o pipeline descobriu sobre ele: metadados, capa, sumário, descrição por IA, estado do processamento e vetor de busca. O texto completo fica em `library_documentcontent`.

Aponta para `owner_id → accounts_user.id`. Referenciada por `library_documentcontent`, `library_documenttag`, `library_collectionitem`, `library_readingstate`, `library_annotation` e `processing_job` (todas pela coluna `document_id`).

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	uuid	não	uuid4()	Identificador. Também é o nome da pasta no disco: /data/documents/<id>/.
owner_id FK	uuid	não	—	Quem enviou → accounts_user.id.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
title	varchar(300)	não	—	Começa com o nome do arquivo e é trocado pelo título dos metadados quando ele é bom (e o usuário não editou).
author	varchar(200)	não	''	Autor dos metadados ou digitado pelo usuário.
description	text	não	''	Descrição escrita pelo usuário (opcional). Entra na busca.
metadata_locked	boolean	não	false	true quando título ou autor foram editados à mão: os metadados do arquivo deixam de sobrescrever.
summary	text	não	''	Descrição gerada pela IA (2 a 3 frases).
summary_source	varchar(20)	não	''	Quem escreveu: local, anthropic ou openai.
summary_model	varchar(100)	não	''	Modelo usado (vazio no resumo local).
summary_at	timestamptz	sim	—	Quando a descrição foi gerada.
kind	varchar(8)	não	—	pdf ou docx, detectado pelo conteúdo do arquivo (não pela extensão).
original_name	varchar(255)	não	—	Nome do arquivo enviado; usado no download.
size_bytes	bigint	não	—	Tamanho em bytes.
sha256 UK¹	varchar(64)	não	—	Impressão digital do arquivo. Com owner_id, impede enviar o mesmo arquivo duas vezes.
page_count	integer	sim	—	Páginas (PDF) ou estimativa (DOCX: app.xml ou palavras ÷ 350).
word_count	integer	sim	—	Palavras do texto extraído.
language	varchar(16)	não	''	Idioma dos metadados (pt-BR) ou detectado no texto (pt, en, es).
cover_width	integer	sim	—	Largura da capa gerada, em pixels.
cover_height	integer	sim	—	Altura da capa gerada, em pixels.
cover_color	varchar(7)	não	''	Cor média da capa (#rrggbb). Fundo do cartão enquanto a imagem carrega.
cover_generated	boolean	não	false	true para a capa tipográfica dos DOCX.
cover_version	integer	não	0	Sobe a cada capa nova; 0 = sem capa. Vai no endereço da imagem (?v=) para renovar o cache.
has_reader_html	boolean	não	false	true quando existe reader.safe.html (DOCX).
outline	jsonb	não	[]	Sumário do documento (até 500 itens).
metadata	jsonb	não	{}	Metadados técnicos do arquivo devolvidos pelo processador.
status	varchar(12)	não	'queued'	queued, extracting, describing, ready ou failed (capítulo 7).
status_detail	text	não	''	Motivo da falha (até 1.000 caracteres) ou aviso de nova tentativa (“Nova tentativa em 15 s”).
attempts	smallint	não	0	Tentativas da etapa atual. Volta a 0 quando a etapa termina.
visibility	varchar(10)	não	'private'	private (só o dono) ou shared (todas as contas da instalação).
search_vector	tsvector	sim	—	Vetor da busca em texto completo, calculado pela aplicação (capítulo 8).

BANCO DE DADOS

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
created_at	timestamptz	não	now()	Envio.
updated_at	timestamptz	não	now()	Última alteração (o Django atualiza a cada `save()`; `UPDATE` em lote não mexe nela).
processed_at	timestamptz	sim	—	Primeira vez em que o documento ficou pronto.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
library_document_pkey	PRIMARY KEY (id)	Chave primária.
library_document_owner_sha256_unique	UNIQUE (owner_id, sha256)	Deduplicação por dono.
library_doc_owner_created	btree (owner_id, created_at DESC)	Biblioteca de uma pessoa, mais recentes primeiro.
library_doc_shared	btree (visibility, created_at DESC) WHERE visibility = 'shared'	Índice parcial: só os compartilhados.
library_doc_search	GIN (search_vector)	Busca em texto completo.
library_doc_title_trgm	GIN (title gin_trgm_ops)	Similaridade por trigramas no título (erros de digitação).
library_document_status_…	btree (status)	Painel Sistema e varredura de documentos parados.
library_document_status_…_like	btree (status varchar_pattern_ops)	Criado pelo Django junto com o índice de `status`.
library_document_owner_id_…	btree (owner_id)	Chave estrangeira.

VERIFICAÇÕES `attempts`, `page_count`, `word_count`, `cover_width`, `cover_height` e `cover_version` ≥ 0 (restrições `library_document_<coluna>_check`). [1] Único em conjunto com `owner_id`.

EXEMPLOS DE JSON

`outline (PDF)` Marcadores do PDF com a página de cada um; **`level`** vai de 1 a 4.

```
[{"title": "Sobre este material", "page": 2, "level": 1, "anchor": null},
 {"title": "Tipos mais usados", "page": 5, "level": 2, "anchor": null}]
```

`outline (DOCX)` Títulos do Word; **`anchor`** é o id do título no HTML de leitura.

```
[{"title": "1. Objetivo", "page": null, "level": 1, "anchor": "s-2"},
 {"title": "Jornada e horários", "page": null, "level": 2, "anchor": "s-6"}]
```

`metadata` Só as chaves que vieram preenchidas do processador Java.

```
{"subject": "Apresentação de resultados trimestrais",
 "keywords": "resultados, trimestre, vendas",
 "producer": "Skia/PDF", "creator": "Documento de exemplo",
 "created_at": "2026-09-15T13:00:00Z", "modified_at": "2026-10-02T15:00:00Z",
 "page_width": 960.0, "page_height": 540.0, "encrypted": false,
 "char_count": 2381, "text_truncated": false, "elapsed_ms": 214, "warnings": []}
```

FICHA 04 BIBLIOTECA ACERVO — 4 colunas · 1 índices

library_documentcontent

Texto completo extraído, separado da tabela principal para manter as listagens leves (no máximo uma linha por documento).

Aponta para `document_id → library_document.id` (relação 1 : 0..1).

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
document_id PK FK	uuid	não	—	Documento → `library_document.id`. É chave primária e estrangeira ao mesmo tempo.
text	text	não	`''`	Texto extraído. No PDF, as páginas são separadas pelo caractere `\f` (form feed). Limite do processador: 2 milhões de caracteres (`PROCESSOR_MAX_TEXT_CHARS`).
text_truncated	boolean	não	`false`	`true` quando o texto passou do limite e foi cortado.
updated_at	timestamptz	não	`now()`	Última extração.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
library_documentcontent_pkey	PRIMARY KEY (document_id)	Chave primária e estrangeira; não há índice extra.

FICHA 05 BIBLIOTECA ORGANIZAÇÃO — 5 colunas · 5 índices

library_tag

Etiquetas de cada pessoa. A mesma etiqueta pode ser usada em vários documentos do dono; etiquetas que ficam sem nenhum documento são apagadas pela aplicação.

Aponta para `owner_id → accounts_user.id`. Referenciada por `library_documenttag.tag_id`.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	bigint	não	identity	Identificador.
owner_id FK	uuid	não	—	Dono → `accounts_user.id`.
name	varchar(40)	não	—	Nome em minúsculas, com espaços normalizados (ex.: `banco de dados`).
slug UK¹	varchar(48)	não	—	Versão para endereços (`banco-de-dados`). Única por dono.
created_at	timestamptz	não	`now()`	Criação.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
library_tag_pkey	PRIMARY KEY (id)	Chave primária.
library_tag_owner_slug_unique	UNIQUE (owner_id, slug)	Uma etiqueta por nome e dono.
library_tag_slug_…	btree (slug)	Filtro `?tag=` da biblioteca (`SlugField` é indexado por padrão).
library_tag_slug_…_like	btree (slug varchar_pattern_ops)	Criado pelo Django junto com o índice de `slug`.
library_tag_owner_id_…	btree (owner_id)	Chave estrangeira.

VERIFICAÇÕES [1] Único em conjunto com `owner_id`.

FICHA 06 BIBLIOTECA ORGANIZAÇÃO — 5 colunas · 4 índices

library_documenttag

Liga documentos e etiquetas (N:N) e guarda quem sugeriu a ligação: a IA ou a pessoa. Cada documento recebe no máximo 12 etiquetas por gravação.

Aponta para `document_id → library_document.id` e `tag_id → library_tag.id`.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	bigint	não	identity	Identificador.
document_id FK	uuid	não	—	→ library_document.id.
tag_id FK	bigint	não	—	→ library_tag.id.
source	varchar(10)	não	'usuario'	ia (sugerida na descrição) ou usuario. Gerar a descrição de novo troca só as da IA; salvar as etiquetas pela interface substitui todas.
created_at	timestamptz	não	now()	Criação.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
library_documenttag_pkey	PRIMARY KEY (id)	Chave primária.
library_documenttag_unique	UNIQUE (document_id, tag_id)	A mesma etiqueta não entra duas vezes no documento.
library_documenttag_document_id_…	btree (document_id)	Chave estrangeira.
library_documenttag_tag_id_…	btree (tag_id)	Chave estrangeira.

FICHA 07 BIBLIOTECA ORGANIZAÇÃO — 6 colunas · 3 índices

library_collection

Coleções criadas pelo usuário para agrupar documentos; um documento pode estar em várias.

Aponta para `owner_id → accounts_user.id`. Referenciada por `library_collectionitem.collection_id`.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	uuid	não	uuid4()	Identificador.
owner_id FK	uuid	não	—	Dono → accounts_user.id.
name UK[1]	varchar(80)	não	—	Nome. Único por dono, sem diferenciar maiúsculas.
description	varchar(300)	não	''	Descrição opcional.
created_at	timestamptz	não	now()	Criação.
updated_at	timestamptz	não	now()	Última alteração (inclui documentos adicionados).

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
library_collection_pkey	PRIMARY KEY (id)	Chave primária.
library_collection_owner_name_unique	UNIQUE INDEX (owner_id, lower(name))	Sem duas coleções com o mesmo nome.

BANCO DE DADOS

NOME	DEFINIÇÃO	PARA QUÊ
library_collection_owner_id_…	btree (owner_id)	Chave estrangeira.

VERIFICAÇÕES [1] Único em conjunto com `owner_id`, sobre `lower(name)`.

FICHA 08 BIBLIOTECA ORGANIZAÇÃO 5 colunas · 4 índices

library_collectionitem

Documentos de cada coleção, na ordem escolhida ao arrastar os cartões. É a tabela intermediária do `ManyToManyField Collection.documents`.

Aponta para `collection_id → library_collection.id` e `document_id → library_document.id`.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	bigint	não	identity	Identificador.
collection_id FK	uuid	não	—	→ library_collection.id.
document_id FK	uuid	não	—	→ library_document.id (pode ser um documento compartilhado de outra pessoa).
position	integer	não	0	Ordem dentro da coleção (≥ 0). Um item novo entra depois do último.
added_at	timestamptz	não	now()	Quando entrou na coleção.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
library_collectionitem_pkey	PRIMARY KEY (id)	Chave primária.
library_collectionitem_unique	UNIQUE (collection_id, document_id)	O documento entra uma vez em cada coleção.
library_collectionitem_collection_id_…	btree (collection_id)	Chave estrangeira.
library_collectionitem_document_id_…	btree (document_id)	Chave estrangeira.

VERIFICAÇÕES `position` ≥ 0 (`library_collectionitem_position_check`).

FICHA 09 BIBLIOTECA LEITURA 12 colunas · 5 índices

library_readingstate

O estado de leitura de cada pessoa em cada documento: estante, página, progresso, favorito e última abertura. É pessoal: num documento compartilhado, cada leitor tem a sua linha.

Aponta para `user_id → accounts_user.id` e `document_id → library_document.id`.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	bigint	não	identity	Identificador.
user_id FK	uuid	não	—	Leitor → accounts_user.id.
document_id FK	uuid	não	—	→ library_document.id.
shelf	varchar(10)	não	''	Estante: vazio, para_ler, lendo ou lido (capítulo 7).
page	integer	não	1	Página atual (PDF), limitada ao número de páginas.
progress	double precision	não	0	Fração lida, de 0 a 1 (restrição de verificação).

BANCO DE DADOS

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
location	jsonb	não	{}	Posição no DOCX (bloco do texto).
favorite	boolean	não	false	Marcado com estrela.
started_at	timestamptz	sim	–	Quando foi para “lendo” pela primeira vez.
finished_at	timestamptz	sim	–	Quando foi marcado como lido; limpo ao mudar para outra estante.
last_opened_at	timestamptz	sim	–	Última abertura no leitor (alimenta o “Continuar lendo”).
updated_at	timestamptz	não	now()	Última alteração.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
library_readingstate_pkey	PRIMARY KEY (id)	Chave primária.
library_readingstate_unique	UNIQUE (user_id, document_id)	Uma linha por leitor e documento.
library_reading_recent	btree (user_id, last_opened_at DESC)	“Continuar lendo” e ordenação por leitura recente.
library_readingstate_user_id_…	btree (user_id)	Chave estrangeira.
library_readingstate_document_id_…	btree (document_id)	Chave estrangeira.

VERIFICAÇÕES `progress` entre 0 e 1 (`library_readingstate_progress`); `page` ≥ 0 (`library_readingstate_page_check`).

EXEMPLOS DE JSON

`location` Só no DOCX: índice do bloco (atributo **`data-b`** do HTML) que estava no topo da tela.

```
{"block": 12}
```

FICHA 10 BIBLIOTECA LEITURA — 11 colunas · 4 índices

library_annotation

Destaques, notas e marcadores feitos no leitor. Também são pessoais.

Aponta para `user_id → accounts_user.id` e `document_id → library_document.id`.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	uuid	não	uuid4()	Identificador.
user_id FK	uuid	não	–	Autor → accounts_user.id.
document_id FK	uuid	não	–	→ library_document.id.
kind	varchar(10)	não	–	highlight (destaque), note (nota) ou bookmark (marcador).
page	integer	sim	–	Página do PDF. Nulo no DOCX.
anchor	jsonb	não	{}	Onde a anotação fica: retângulos na página (PDF) ou bloco do texto (DOCX). Até 8.000 caracteres.
quote	text	não	''	Trecho selecionado (até 4.000 caracteres).
note	text	não	''	Texto da nota (até 8.000 caracteres).
color	varchar(16)	não	'salvia'	salvia, rose, ardosia ou camurca.
created_at	timestamptz	não	now()	Criação.
updated_at	timestamptz	não	now()	Última alteração.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
library_annotation_pkey	PRIMARY KEY (id)	Chave primária.
library_annotation_user_doc	btree (user_id, document_id)	Anotações de uma pessoa num documento (abertura do leitor).
library_annotation_user_id_…	btree (user_id)	Chave estrangeira.
library_annotation_document_id_…	btree (document_id)	Chave estrangeira.

VERIFICAÇÕES `page` ≥ 0 (`library_annotation_page_check`). Os limites de tamanho e de cor são da API, que também limita 2.000 anotações por pessoa em cada documento.

EXEMPLOS DE JSON

`anchor (PDF)` Retângulos do trecho em frações da página (x, y, largura, altura), para funcionar em qualquer zoom.

```
{"rects": [[0.099, 0.119, 0.802, 0.0175],
           [0.099, 0.1385, 0.18, 0.0175]]}
```

`anchor (DOCX)` Bloco do texto onde o trecho começa; o destaque é refeito procurando **`quote`** dentro dele.

```
{"block": 7}
```

`anchor (marcador)` Marcador de página não precisa de âncora (a página está em **`page`**).

```
{}
```

FICHA 11 **PROCESSAMENTO** PIPELINE — 11 colunas · 4 índices

processing_job

Histórico de cada tentativa de cada etapa do pipeline. Alimenta o painel Sistema e ajuda a investigar falhas.

Aponta para `document_id → library_document.id`.

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id PK	bigint	não	identity	Identificador.
document_id FK	uuid	não	—	→ library_document.id.
stage	varchar(10)	não	—	extract (leitura pelo processador Java) ou describe (descrição por IA).
status	varchar(10)	não	'running'	running, succeeded, retrying ou failed.
attempt	smallint	não	1	Número da tentativa (cópia de library_document.attempts no início da etapa).
worker	varchar(120)	não	''	Consumidor que executou: <host>-<pid>-<thread>.
started_at	timestamptz	não	now()	Início.
finished_at	timestamptz	sim	—	Fim.
duration_ms	integer	sim	—	Duração em milissegundos.
error	text	não	''	Erro (até 4.000 caracteres) ou aviso, como a queda para o resumo local.
details	jsonb	não	{}	Números da etapa: páginas, palavras, tempo no processador, provedor e modelo de IA, espera da nova tentativa.

ÍNDICES E RESTRIÇÕES

NOME	DEFINIÇÃO	PARA QUÊ
processing_job_pkey	PRIMARY KEY (id)	Chave primária.
processing_job_document	btree (document_id, started_at DESC)	Histórico de um documento.
processing_job_status	btree (status, started_at DESC)	Painel: últimas etapas e falhas.
processing_job_document_id_…	btree (document_id)	Chave estrangeira.

VERIFICAÇÕES `attempt` ≥ 0 e `duration_ms` ≥ 0 (`processing_job_<coluna>_check`).

EXEMPLOS DE JSON

`details (extract)` Leitura concluída.

```
{"pages": 8, "words": 1028, "processor_ms": 442}
```

`details (retrying)` Espera até a próxima tentativa.

```
{"retry_in_s": 15.0}
```

`details (describe)` **`fallback`** é **`true`** quando houve aviso (por exemplo, o provedor externo falhou e o resumo local foi usado).

```
{"provider": "anthropic", "model": "claude-haiku-5-5", "fallback": false}
```

CAPÍTULO 07

Máquinas de estado

Três colunas seguem um ciclo de vida bem definido: o estado do documento, o estado de cada job e a estante de cada leitor.

Documento · `library_document.status`

A coluna segue o pipeline. Setas tracejadas são caminhos de volta: retentativa, ações do dono no detalhe do documento e a varredura de documentos parados.

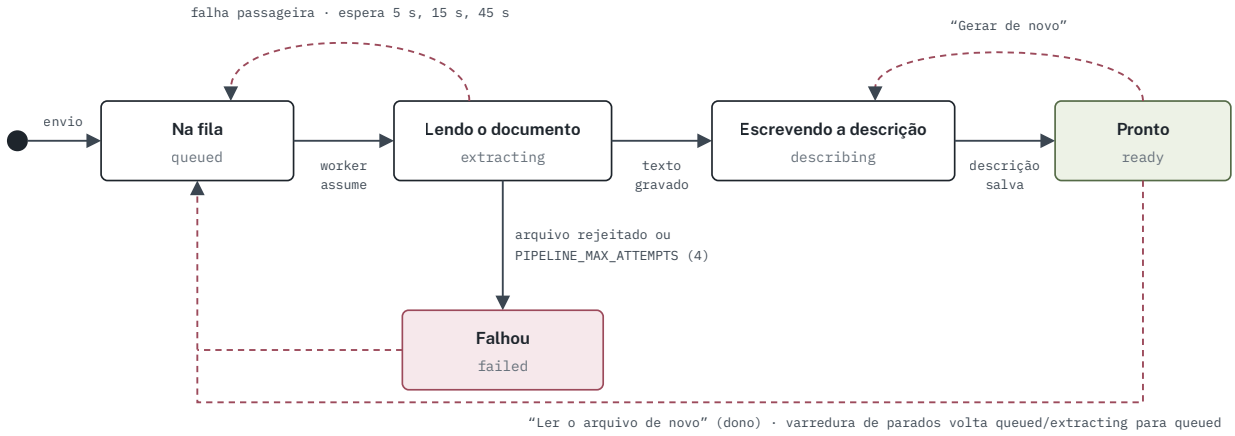


FIGURA 2 Estados do documento.

TRANSIÇÃO	QUEM FAZ	O QUE MUDA
→ `queued`	Envio (web)	Linha criada; arquivo na pasta; depois do `COMMIT`, `XADD docs:extract`.
`queued` → `extracting`	Worker	`attempts + 1`, `status_detail = ''`; cria o job.
`extracting` → `describing`	Worker	Metadados, capa, sumário, texto e busca gravados; `attempts = 0`; `XADD docs:describe`.
`extracting` → `queued`	Worker	Erro passageiro (processador fora do ar, sem vaga, rede): `status_detail = 'Nova tentativa em N s'` e entrada no ZSET `docs:delayed`. Espera de 5 s × $3^{\text{tentativa}-1}$, no máximo 300 s: com o padrão de 4 tentativas, 5 s, 15 s e 45 s.
`extracting` → `failed`	Worker	Arquivo rejeitado (senha, corrompido, formato) ou `PIPELINE_MAX_ATTEMPTS` tentativas sem sucesso (padrão 4).
`describing` → `ready`	Worker	Descrição, etiquetas da IA, idioma e busca; `processed_at` na primeira vez. Se a IA externa falhar, usa o resumo local: esta etapa não falha.
`ready` / `failed` → `queued`	Dono: "Ler o arquivo de novo"	`attempts = 0`, `status_detail = ''`; volta à fila de leitura.
`ready` → `describing`	Dono: "Gerar de novo"	Volta só para a fila de descrição.
parado → fila	Varredura do worker (a cada 30 s)	Documento em `queued` / `extracting` / `describing` sem mudança há 30 min (`PIPELINE_STALE_AFTER`) volta para a fila da etapa: `describing` continua `describing`; os outros voltam a `queued`. No máximo 200 por rodada.

Job · `processing_job.status`

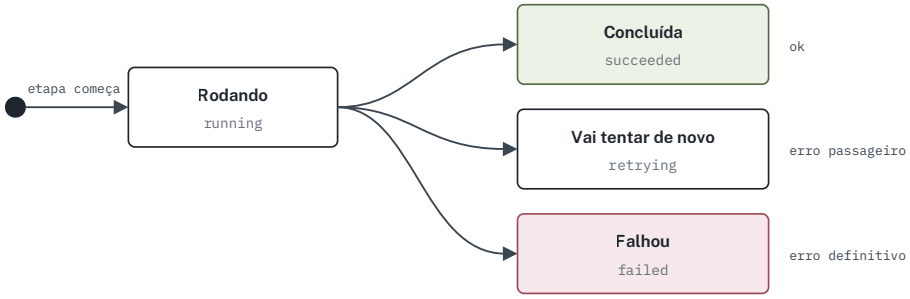


FIGURA 3 Estados de uma tentativa.

Cada tentativa começa como `running`. Ao terminar, grava `finished_at`, `duration_ms`, o erro (se houver) e os `details`. `retrying` significa que o documento voltou para a fila com espera; a próxima tentativa cria outra linha.

Estante · `library_readingstate.shelf`

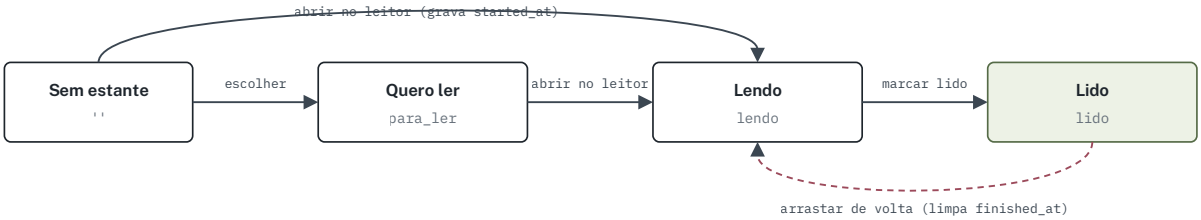


FIGURA 4 Estantes de leitura.

A estante começa vazia. Abrir o documento no leitor move de vazio ou “Quero ler” para “Lendo” e grava `started_at` (só na primeira vez). Marcar como lido grava `finished_at`; mover para outra estante limpa essa data. Na tela Estantes, o usuário arrasta o cartão para qualquer coluna.

CAPÍTULO 08

Busca em texto completo

A busca roda inteira dentro do PostgreSQL: um `tsvector` com pesos, a configuração `pt_unaccent` e trigramas no título.

Cada documento tem um `search_vector` com pesos. O worker o recalcula ao terminar cada etapa; a API recalcula quando o dono edita título, autor, descrição ou etiquetas (função `refresh_search` em `library/services.py`).

PESO	CONTEÚDO
`A`	Título.
`B`	Autor; descrição por IA, descrição do usuário e nomes das etiquetas.
`C`	Texto extraído (primeiros 300 mil caracteres, para respeitar o limite de 1 MB do `tsvector`).

Como o vetor é calculado

```
UPDATE library_document d SET search_vector =
    setweight(to_tsvector('pt_unaccent', coalesce(d.title, '')), 'A') ||
    setweight(to_tsvector('pt_unaccent', coalesce(d.author, '')), 'B') ||
    setweight(to_tsvector('pt_unaccent', coalesce(d.summary, '') || ' ' ||
coalesce(d.description, '') || ' ' ||
        coalesce((SELECT string_agg(t.name, ' ') FROM library_documenttag dt
                  JOIN library_tag t ON t.id = dt.tag_id WHERE dt.document_id = d.id), '')), 'B')
||
    setweight(to_tsvector('pt_unaccent', coalesce(
        (SELECT left(c.text, 300000) FROM library_documentcontent c WHERE c.document_id = d.id),
'')), 'C')
WHERE d.id = :id;
```

Como a busca consulta

A caixa de busca aceita a sintaxe de `websearch_to_tsquery`: palavras soltas, `"frase exata"`, `-palavra` para excluir e `or`. Um documento entra no resultado se casar com a busca textual ou se o título tiver similaridade de trigramas acima de 0,25 (pega erros como “postgress”). A ordem é relevância (`ts_rank`), depois similaridade, depois data de envio. A página tem 48 resultados (até 200 com `page_size`). Equivalente em SQL:

```
SELECT d.id, d.title,
       ts_rank(d.search_vector, websearch_to_tsquery('pt_unaccent', 'banco de dados')) AS rank,
       similarity(d.title, 'banco de dados') AS sim
FROM library_document d
WHERE (d.owner_id = :usuario OR d.visibility = 'shared')
  AND (d.search_vector @@ websearch_to_tsquery('pt_unaccent', 'banco de dados')
       OR similarity(d.title, 'banco de dados') > 0.25)
ORDER BY rank DESC, sim DESC, d.created_at DESC
LIMIT 48;
```

Trechos encontrados

Para os documentos da página atual, a API pede o trecho onde a busca aparece. O texto é escapado antes de virar HTML e só as marcações `<mark>` são acrescentadas (os delimitadores internos `@@@[` e `]@@@` são trocados por elas).

```
SELECT c.document_id,
       ts_headline('pt_unaccent', left(c.text, 100000),
                   websearch_to_tsquery('pt_unaccent', 'banco de dados'),
                   'MaxFragments=1, MaxWords=26, MinWords=12, StartSel=@@@[, StopSel=]@@@,
FragmentDelimiter= … ')
FROM library_documentcontent c
WHERE c.document_id = ANY(:ids);
```

> **POR QUE PT_UNACCENT**
>
> A configuração tira os acentos e reduz as palavras ao radical português: decisões, decisao e decisão se aproximam. Sem ela, quem digita sem acento não acha nada.

CAPÍTULO 09

Redis: filas, chaves e eventos

O caminho de um documento do envio até “Pronto”, com cada leitura e escrita no PostgreSQL e no Redis, e a lista completa das chaves.

Um documento passo a passo

1. Envio

01 O web grava o arquivo em `/data/tmp/<uuid>.part` calculando o SHA-256 e checa o tamanho (`APP_MAX_UPLOAD_MB`, padrão 200).

02 Detecta o tipo pelo conteúdo (`%PDF-` ou ZIP com `word/document.xml`). Arquivos `.doc` antigos são recusados.

03 Se já existe `(owner_id, sha256)`, devolve o documento existente e apaga o temporário.

04 Numa transação: `INSERT library_document` (estado `queued`, título a partir do nome do arquivo), move o arquivo para `documents/<id>/original.<tipo>` e, se veio uma coleção, `INSERT library_collectionitem`.

05 Depois do `COMMIT`: `XADD docs:extract` e `PUBLISH events:user:<dono>` (`document.created`).

2. Leitura do arquivo (processador Java)

01 Um worker lê a mensagem com `XREADGROUP` no grupo `workers` e assume o documento: `status = 'extracting'`, `attempts + 1`.

02 `INSERT processing_job` (etapa `extract`, estado `running`) e aviso pelo pub/sub.

03 Chama `POST /v1/process` no processador, que escreve `cover.jpg`, `reader.html` e `media/` na pasta.

04 Numa transação: atualiza páginas, palavras, idioma, sumário, metadados, capa (`cover_version + 1`), título e autor (se não estiverem travados), `status = 'describing'`; grava o texto em `library_documentcontent` (insere ou atualiza); recalcula o `search_vector`. No DOCX, higieniza o HTML e grava `reader.safe.html`.

05 Depois do `COMMIT`, `XADD docs:describe`; fecha o job como `succeeded` e faz `XACK` da mensagem.

3. Descrição por IA

01 Outro (ou o mesmo) worker assume pela fila `docs:describe` e cria o job `describe`.

02 Lê o texto, pede a descrição ao provedor configurado (local, Anthropic ou API compatível com OpenAI) e, se ele falhar, usa o resumo local.

03 Numa transação: grava `summary`, `summary_source`, `summary_model`, `summary_at`, idioma (se ainda vazio), `status = 'ready'`, `processed_at`; troca as etiquetas de origem `ia` (as do usuário ficam); recalcula a busca.

04 Fecha o job e publica o evento: os navegadores abertos atualizam o cartão e a capa aparece.

Chaves do Redis

CHAVE	TIPO	PARA QUÊ
`docs:extract`	stream	Fila da leitura do arquivo. Campo `document_id`; tamanho aproximado limitado a 10 mil entradas (`MAXLEN ~ 10000`).
`docs:describe`	stream	Fila da descrição por IA. Mesmo formato e limite.
`workers`	grupo de consumidores	Grupo das duas filas, criado com `XGROUP CREATE … MKSTREAM` e recriado se sumir. Mensagem sem confirmação há mais de 10 min (`PIPELINE_VISIBILITY_TIMEOUT`) é reassumida com `XAUTOCLAIM`, publicada de novo e confirmada.
`docs:delayed`	sorted set	Retentativas com espera; membro JSON `{"stage", "document_id"}`, score = horário (epoch). Os workers movem os vencidos para o stream (até 100 por vez).
`events:user:<id>`	pub/sub	Avisos para os navegadores daquela conta.
`events:shared`	pub/sub	Avisos de documentos compartilhados, para todas as contas.
`workers:<host>-<pid>`	hash, TTL 30 s	Batimento de cada worker: `name`, `stages`, `threads`, `busy`, `processed`, `failed`, `started`, `seen`. Some sozinho se o worker morrer; é apagado ao encerrar.
`login:fail:<ip>:<email>`	string, TTL 15 min	Tentativas de login erradas; a partir de 5, o login fica bloqueado até expirar.
`cache:1:*`	strings	Cache do Django (`RedisCache`, prefixo `cache`, versão 1), incluindo a cópia das sessões.

Para inspecionar: `docker compose exec redis redis-cli XINFO GROUPS docs:extract`, `XPENDING docs:extract workers`, `ZCARD docs:delayed` e `KEYS workers:*`. O painel Sistema mostra os mesmos números (tamanho, pendentes, aguardando e atrasados).

Eventos em tempo real

Cada processo web mantém uma única assinatura `PSUBSCRIBE events:*` e distribui as mensagens às conexões WebSocket abertas; cada conexão escuta `events:user:<id>` e `events:shared`. Não há camada de canais do Channels no Redis (`CHANNEL_LAYERS` vazio). A mensagem só avisa o que mudou; o navegador busca o documento de novo pela API, com as próprias permissões.

EVENTO	CANAL	CONTEÚDO E QUANDO
`document.created`	usuário (+ shared)	`{"type", "id", "status", "owner_id"}` — depois do envio.
`document.updated`	usuário (+ shared)	Mesmo formato — a cada mudança de estado ou edição.
`document.deleted`	usuário (+ shared)	`{"type", "id"}` — exclusão; também quando um documento deixa de ser compartilhado (com `only_shared: true`, ignorado pelo dono).
`reading.updated`	usuário	`{"type", "id"}` — estante, página ou favorito alterados.
`collections.changed`	usuário	`{"type"}` — coleção criada, alterada ou excluída, ou itens mudados.

“+ shared” significa que o evento também vai para `events:shared` quando o documento é compartilhado. Uma fila por conexão guarda até 200 avisos; um cliente lento perde avisos, mas recarrega ao reconectar.

CAPÍTULO 10

Consultas úteis para operação

Consultas de diagnóstico, somente leitura, escritas para o esquema real. Para abrir o `psql` no Docker: `docker compose exec postgres psql -U lume -d lume`.

Pipeline

Documentos por estado, com o mais antigo de cada um

```
SELECT status, count(*) AS documentos, min(updated_at) AS mais_antigo
FROM library_document
GROUP BY status
ORDER BY status;
```

Documentos parados há mais de 30 minutos

```
SELECT id, title, status, attempts, status_detail,
       now() - updated_at AS parado_ha
FROM library_document
WHERE status IN ('queued', 'extracting', 'describing')
  AND updated_at < now() - interval '30 minutes'
ORDER BY updated_at;
```

Fila de falhas: documentos que falharam e o último erro

```
SELECT d.id, d.title, u.email AS dono, d.status_detail,
       j.stage, j.attempt, j.error, j.started_at
FROM library_document d
JOIN accounts_user u ON u.id = d.owner_id
LEFT JOIN LATERAL (
    SELECT stage, attempt, error, started_at
    FROM processing_job
    WHERE document_id = d.id
    ORDER BY started_at DESC
    LIMIT 1
) j ON true
WHERE d.status = 'failed'
ORDER BY d.updated_at DESC;
```

Falhas e retentativas recentes

```
SELECT d.title, j.stage, j.status, j.attempt, j.error, j.started_at
FROM processing_job j
JOIN library_document d ON d.id = j.document_id
WHERE j.status IN ('failed', 'retrying')
ORDER BY j.started_at DESC
LIMIT 20;
```

Tempo por etapa nas últimas 24 horas

```
SELECT stage, count(*) AS etapas, round(avg(duration_ms)) AS media_ms,
       percentile_disc(0.95) WITHIN GROUP (ORDER BY duration_ms) AS p95_ms
FROM processing_job
WHERE status = 'succeeded' AND started_at > now() - interval '24 hours'
GROUP BY stage;
```

Descrições feitas pelo resumo local por falha do provedor

```
SELECT details->>'provider' AS provedor, count(*) AS etapas,
       count(*) FILTER (WHERE (details->>'fallback')::boolean) AS com_aviso
FROM processing_job
WHERE stage = 'describe' AND status = 'succeeded'
  AND started_at > now() - interval '7 days'
GROUP BY 1;
```

Acervo e uso

Maiores arquivos

```
SELECT d.title, d.kind, pg_size_pretty(d.size_bytes) AS tamanho, u.email AS dono
FROM library_document d
JOIN accounts_user u ON u.id = d.owner_id
ORDER BY d.size_bytes DESC
LIMIT 10;
```

Espaço ocupado por conta

```
SELECT u.email, count(d.id) AS documentos,
       pg_size_pretty(coalesce(sum(d.size_bytes), 0)) AS arquivos
FROM accounts_user u
LEFT JOIN library_document d ON d.owner_id = u.id
GROUP BY u.email
ORDER BY coalesce(sum(d.size_bytes), 0) DESC;
```

Textos cortados pelo limite do processador

```
SELECT d.id, d.title, length(c.text) AS caracteres
FROM library_documentcontent c
JOIN library_document d ON d.id = c.document_id
WHERE c.text_truncated;
```

Leitura por pessoa

```
SELECT u.name,
       count(*) FILTER (WHERE r.shelf = 'lendo') AS lendo,
       count(*) FILTER (WHERE r.shelf = 'lido')  AS lidos,
       count(*) FILTER (WHERE r.favorite)        AS favoritos,
       max(r.last_opened_at)                     AS ultima_leitura
FROM library_readingstate r
JOIN accounts_user u ON u.id = r.user_id
GROUP BY u.name
ORDER BY ultima_leitura DESC NULLS LAST;
```

Etiquetas mais usadas

```
SELECT t.name, count(*) AS documentos,
       count(*) FILTER (WHERE dt.source = 'ia') AS sugeridas_pela_ia
FROM library_documenttag dt
JOIN library_tag t ON t.id = dt.tag_id
GROUP BY t.name
ORDER BY documentos DESC
LIMIT 20;
```

Tokens de API válidos

```
SELECT u.email, t.name, t.prefix, t.last_used_at, t.expires_at
FROM accounts_apitoken t
JOIN accounts_user u ON u.id = t.user_id
WHERE t.revoked_at IS NULL
  AND (t.expires_at IS NULL OR t.expires_at > now())
ORDER BY t.last_used_at DESC NULLS LAST;
```

Saúde do banco

Tamanho das tabelas (com índices e TOAST)

```
SELECT relname AS tabela,
       pg_size_pretty(pg_total_relation_size(relid)) AS total,
       pg_size_pretty(pg_relation_size(relid))       AS dados,
       pg_size_pretty(pg_indexes_size(relid))        AS indices
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC;
```

Linhas mortas e última limpeza automática

```
SELECT relname, n_live_tup, n_dead_tup, last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;
```

Índices pouco usados

```
SELECT relname AS tabela, indexrelname AS indice, idx_scan,
       pg_size_pretty(pg_relation_size(indexrelid)) AS tamanho
FROM pg_stat_user_indexes
ORDER BY idx_scan, pg_relation_size(indexrelid) DESC
LIMIT 15;
```

CAPÍTULO 11

Como preparar o banco

Com Docker não há nada a fazer. Com um PostgreSQL próprio ou gerenciado, basta criar o usuário, o banco e as duas extensões antes da primeira migração.

Com Docker (recomendado)

O `docker compose up` sobe o PostgreSQL 16 com o usuário e o banco de `DB_USER`, `DB_NAME` e `DB_PASSWORD` (padrão `lume` / `lume`; o `iniciar.cmd` / `iniciar.sh` gera a senha), e o serviço `web` roda `python manage.py migrate` antes de abrir a porta. Como o usuário criado pela imagem é superusuário do banco, a migração cria as extensões sozinha. A porta 5432 só é publicada em `127.0.0.1`.

PostgreSQL próprio

```
-- como superusuário (postgres)
CREATE ROLE lume LOGIN PASSWORD 'uma-senha-forte';
CREATE DATABASE lume OWNER lume ENCODING 'UTF8';
\c lume
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS unaccent;
```

Criar as extensões antes é necessário quando o usuário da aplicação não é superusuário (a migração usa `CREATE EXTENSION IF NOT EXISTS` e passa direto se elas já existem). Depois, aponte a aplicação para o banco:

VARIÁVEL	PADRÃO FORA DO DOCKER	NO DOCKER	USO
`DB_HOST`	`localhost`	`postgres`	Servidor.
`DB_PORT`	`5432`	`5432`	Porta.
`DB_NAME`	`library`	`lume`	Banco.
`DB_USER`	`library`	`lume`	Usuário.
`DB_PASSWORD`	vazio	obrigatória	Senha.
`DB_CONN_MAX_AGE`	`60`	`60`	Segundos que uma conexão fica aberta para reuso.
`REDIS_URL`	`redis://localhost:6379/0`	`redis://redis:6379/0`	Filas, eventos e cache.

> ATENÇÃO AOS PADRÕES
>
> Fora do Docker, o nome do banco e do usuário caem para `library` se `DB_NAME` e `DB_USER` não forem definidos. Para usar os comandos deste documento como estão, defina os dois como `lume`.

```
cd web
python manage.py migrate
python manage.py createuser --email voce@empresa.com --name "Seu Nome" --admin
```

Banco gerenciado

Serviços como Amazon RDS, Google Cloud SQL, Azure Database ou Supabase funcionam: `pg_trgm` e `unaccent` estão na lista de extensões permitidas de todos eles. Crie as extensões com o usuário administrador do serviço antes da primeira migração, use conexão com TLS quando o provedor exigir e lembre que a busca roda inteira dentro do PostgreSQL (não há serviço de busca à parte).

Redis

Redis 7 (`REDIS_URL` , padrão `redis://localhost:6379/0`). No Docker ele roda com `appendonly yes` para não perder filas num reinício. Sem persistência também funciona: os workers recriam os grupos e a varredura devolve à fila o que estava pendente. Os testes usam o banco 15 do Redis (`TEST_REDIS_URL`) e o apagam a cada teste; nunca aponte os testes para o Redis de produção.

CAPÍTULO 12

Manutenção, backup e restauração

Rotinas para manter o banco saudável, copiar e recuperar os dados, e resolver os problemas mais comuns.

VACUUM, ANALYZE e índices

O autovacuum do PostgreSQL vem ligado e dá conta do uso normal. As tabelas que mais recebem `UPDATE` são `library_document` (cada etapa regrava a linha e o vetor de busca) e `library_readingstate` (página e progresso a cada leitura). Rode à mão depois de importações grandes, de uma restauração ou de um expurgo de histórico:

```
# estatísticas e limpeza de uma tabela
docker compose exec postgres psql -U lume -d lume -c "VACUUM (ANALYZE, VERBOSE)
library_document;"

# todas as tabelas, em estágios (útil logo depois de um pg_restore)
docker compose exec postgres vacuumdb -U lume -d lume --analyze-in-stages

# reconstruir o índice de busca sem bloquear leituras
docker compose exec postgres psql -U lume -d lume -c "REINDEX INDEX CONCURRENTLY
library_doc_search;"
```

Evite `VACUUM FULL` com o Lume no ar: ele bloqueia a tabela inteira enquanto a reescreve. Para recuperar espaço depois de apagar muitos documentos, pare `web` e `worker` antes.

Retenção e tamanho

- O Lume não apaga nada sozinho. O que mais cresce é `library_documentcontent` (texto completo, até 2 milhões de caracteres por documento) e `processing_job` (duas linhas por documento processado, mais as retentativas e reprocessamentos).
- O histórico de jobs pode ser enxugado sem efeito no funcionamento: `DELETE FROM processing_job WHERE started_at < now() - interval '90 days';` seguido de `VACUUM (ANALYZE) processing_job;`
- O tamanho de cada tabela está na consulta do capítulo 10. O volume de arquivos ocupa mais ou menos a soma de `size_bytes` mais 40 a 80 KB de capa por documento.

Backup

Um backup completo tem duas partes, feitas no mesmo momento: o banco e o volume de arquivos. O nome do volume vem do projeto do Compose (`name: lume`), por isso `lume_documents`.

```
# 1. banco (formato custom, compactado)
docker compose exec -T postgres pg_dump -U lume -d lume -Fc > lume-$(date +%F).dump

# 2. arquivos dos documentos (originais, capas, HTML de leitura)
docker run --rm -v lume_documents:/data -v "$PWD":/backup alpine \
    tar czf /backup/documentos-$(date +%F).tgz -C /data .

# 3. conferir o dump (lista os objetos sem restaurar)
pg_restore --list lume-$(date +%F).dump | head
```

Restauração

```
docker compose stop web worker
docker compose up -d postgres
docker compose exec -T postgres pg_restore -U lume -d lume --clean --if-exists <
lume-2026-10-20.dump
docker run --rm -v lume_documents:/data -v "$PWD":/backup alpine \
    sh -c "cd /data && tar xzf /backup/documentos-2026-10-20.tgz"
docker compose exec postgres vacuumdb -U lume -d lume --analyze-in-stages
docker compose up -d
```

O `pg_dump` leva junto as extensões e a configuração `pt_unaccent`. Se restaurar só o banco, os documentos aparecem sem capa e sem arquivo; nesse caso, use “Ler o arquivo de novo” depois de devolver os originais. O Redis não precisa de backup: ao subir, os workers recriam os grupos e a varredura recoloca na fila o que estava pendente.

Problemas comuns

SINTOMA	CAUSA PROVÁVEL E SOLUÇÃO
`permission denied to create extension "pg_trgm"`	O usuário da aplicação não pode criar extensões. Crie `pg_trgm` e `unaccent` como superusuário e rode `migrate` de novo.
`text search configuration "pt_unaccent" does not exist`	A migração `library.0001` não foi aplicada nesse banco. Rode `python manage.py migrate`.
`password authentication failed for user "library"`	Fora do Docker, `DB_USER` / `DB_NAME` não foram definidos e caíram no padrão `library`. Defina as variáveis.
Documentos parados em “Na fila”	Worker ou processador fora do ar. Veja `docker compose ps` e `docker compose logs worker processor`. Quando voltarem, a fila anda; documentos esquecidos voltam sozinhos em até 30 min.
“Já estava na sua biblioteca” ao enviar	O mesmo arquivo (mesmo SHA-256) já existe para essa conta. É a restrição `(owner_id, sha256)`.
A busca não acha um documento recém-enviado	O vetor é calculado ao fim da leitura. Espere o estado “Pronto”.
`duplicate key value violates unique constraint "accounts_user_email_unique"`	Já existe conta com esse e-mail (com outras maiúsculas).
O Redis foi apagado	Nada a fazer: os workers recriam os grupos e a varredura recoloca os documentos pendentes na fila.

SINTOMA	CAUSA PROVÁVEL E SOLUÇÃO
`DELETE` manual falha com violação de chave estrangeira	As cascatas são feitas pelo Django. Apague pela interface/API ou apague antes as linhas filhas (capítulo 5).

CAPÍTULO 13

Evolução do esquema

O esquema é definido pelos modelos do Django (`web/apps/*/models.py`) e versionado em migrações (`web/apps/*/migrations/`). O banco nunca é alterado à mão.

01 Altere o modelo (por exemplo, uma coluna nova em `Document`).

02 Gere a migração: `python manage.py makemigrations library`. Revise o arquivo gerado.

03 Veja o SQL que será executado: `python manage.py sqlmigrate library 0002`.

04 Aplique: `python manage.py migrate` (no Docker, o serviço `web` aplica sozinho ao subir).

05 Confira o estado: `python manage.py showmigrations`.

06 Rode os testes (`python -m pytest`): eles criam um banco de teste do zero com todas as migrações.

> REGRAS
>
> - Nunca edite uma migração que já foi aplicada em algum ambiente; crie uma nova.
> - Coluna nova em tabela grande: crie como nula ou com valor padrão e preencha em lotes antes de torná-la obrigatória.
> - Índice novo em tabela grande: prefira `AddIndexConcurrently` (de `django.contrib.postgres.operations`) numa migração com `atomic = False`.
> - Mudou o cálculo da busca? Recalcule os vetores depois do deploy (por exemplo, com “Ler o arquivo de novo” ou um comando de manutenção que chame `refresh_search` para cada documento).

Migrações atuais

MIGRAÇÃO	CRIA
`accounts.0001_initial`	`accounts_user` (com `accounts_user_email_unique`) e `accounts_apitoken`.
`library.0001_initial`	Extensões `pg_trgm` e `unaccent`, configuração `pt_unaccent` e as oito tabelas `library_*` com índices e restrições.
`processing.0001_initial`	`processing_job` e seus dois índices; depende de `library.0001`.
Django	Migrações de `auth`, `contenttypes` e `sessions`.

CAPÍTULO 14

Histórico de versões

Versões deste documento e do esquema que ele descreve.

VERSÃO	DATA	ESQUEMA	MUDANÇAS
1.1.0	outubro de 2026	sem mudanças	Documento revisado e reorganizado na nova identidade visual. Acrescentados: visão rápida, dicionário de dados com convenções de nomes e valores fixos, eventos em tempo real, consultas de operação, rotinas de manutenção (VACUUM/ANALYZE, REINDEX) e este histórico. Diagrama de relacionamentos redesenhado. Corrigidos em relação à 1.0.0: lista completa de índices (inclusive os automáticos de chave estrangeira e `_like`), esperas reais das retentativas com o padrão de 4 tentativas, padrões de `DB_NAME` / `DB_USER` fora do Docker e o efeito de salvar etiquetas pela interface.
1.0.0	outubro de 2026	migrações `0001_initial`	Lançamento.

A versão 1.1.0 não altera o esquema: as migrações continuam sendo `accounts.0001_initial`, `library.0001_initial` e `processing.0001_initial`.

COLOFÃO

Lume · Banco de dados · versão 1.1.0 · outubro de 2026. Composto em Literata, Public Sans e IBM Plex Mono; gerado com WeasyPrint a partir de `docs/fonte/banco-de-dados.html`.

Copyright © 2026 Clara A. Portes. Distribuído sob a Licença MIT.